애플리케이션 설계

풀스택 개발자 과정

1조: 박한결, 박영준, 김선길, 원병선

목차

01 구동 확인

02 DataSourceConfig

03 GlobalExceptionHandler

04 MemberController

05 MemberDAO

06 MemberDTO

GitHub: https://github.com/Fullstack-Tests/ApplicationDesign

01 구동 확인

```
6115818 (HEAD -> develop, origin/develop, origin/HEAD) V1.0 [조장] 최종 확인 완료
59aba57 V0.4 Merge pull request #8 from FulstackTest/feature/원병선
1e19575 V0.3 Merge pull request #7 from FulstackTest/feature/박영준
d985d28 V0.2 Merge remote-tracking branch 'origin/feature/김선길' into develop
3ac7cb1 (origin/feature/김선길) V0.4 김선길 최종
297b2d6 V0.1 Merge pull request #5 from FulstackTest/feature/김선길
9f6a081 V0.1 Merge pull request #4 from FulstackTest/feature/박한결
bbbdcc6 (origin/feature/원병선) MemberDAO - 회원 전체 조회
1042025 MemberDAO - 회원 1건 등록
a51b6f7 (origin/feature/박영준) [박영준] V0.4 memberAddPost 유효성 검증 및 회원등록 구현
471f29a [박영준] V0.2 @ExceptionHandler 메서드 작성
e916c96 [박영준] V0.1 HikariDataSource 빈 등록
0530c20 [박영준] V0.0 BasicDataSource(commons-dbcp2) 빈 등록
edbca7f (origin/feature/박한결) [박한결] V0.0 기타 모든 예외 처리
aa38f47 [박한결] V0.0 산술연산 예외 처리
a5bd14c [박한결] V0.0 미존재파일 예외 처리
7f959c8 V0.3 TODO 주석처리
e0d6125 V0.2 애너테이션 추가
8c8a53f V0.1 getter/setter/builder 생성
5a49fd1 V0.0 Lombok과 검증 import 추가
0233b6e (origin/main) V0.0 베이스 업로드
f6969a1 Initial commit
```

commit 확인

01 구동 확인

```sql
DROP TABLE IF EXISTS tbl_member;

CREATE TABLE tbl_member (
    id       BIGINT       NOT NULL AUTO_INCREMENT,
    name     VARCHAR(50),
    email    VARCHAR(100),
    phone    VARCHAR(20),
    createAt TIMESTAMP,
    PRIMARY KEY (id)
);

-- seed 3건
INSERT INTO tbl_member VALUES (null, '김철수', 'kim@example.com', '010-1111-2222', now());
INSERT INTO tbl_member VALUES (null, '이영희', 'lee@example.com', '010-3333-4444', now());
INSERT INTO tbl_member VALUES (null, '박민수', 'park@example.com', '010-5555-6666', now());
```

Result Grid

id	name	email	phone	createAt
1	김철수	kim@example.com	010-1111-2222	2026-06-05 16:09:13
2	이영희	lee@example.com	010-3333-4444	2026-06-05 16:09:13
3	박민수	park@example.com	010-5555-6666	2026-06-05 16:09:13

tbl_member 1

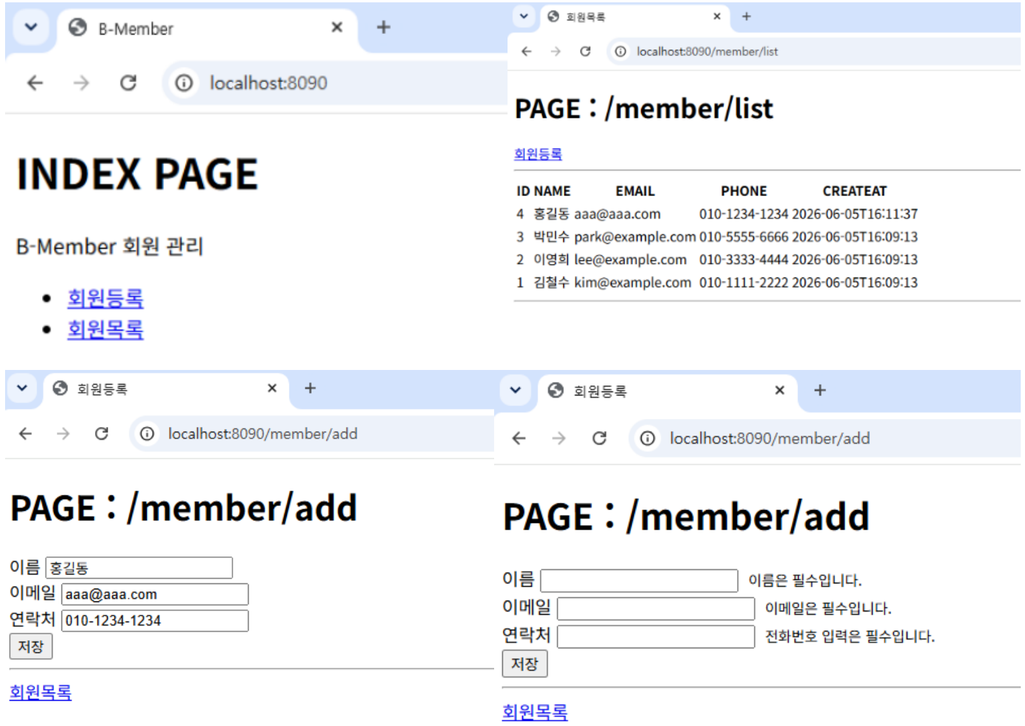
Output

Action Output

#	Time	Action	Message
14	16:09:13	INSERT INTO tbl_member VALUES (null, '김철수', 'kim@example.com', '010-1111-2222', now())	1 row(s) affected
15	16:09:13	INSERT INTO tbl_member VALUES (null, '이영희', 'lee@example.com', '010-3333-4444', now())	1 row(s) affected
16	16:09:13	INSERT INTO tbl_member VALUES (null, '박민수', 'park@example.com', '010-5555-6666', now())	1 row(s) affected
17	16:09:24	SELECT * FROM testdb.tbl_member	3 row(s) returned

tbl_member 테이블 Create

01 구동 확인



구동 확인

1. **회원목록** 페이지에서 회원들의 목록을 확인할 수 있다.
2. **회원등록** 페이지에서 회원의 정보를 작성하여 저장할 수 있다.
3. 아무것도 작성하지 않고 **저장**하려고 하면 검증 메시지가 도출되는 것을 확인할 수 있다.

02 DataSourceConfig - 박영준

```java
package com.example.demo.Config;

import com.zaxxer.hikari.HikariDataSource;
import org.apache.commons.dbcp2.BasicDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.sql.DataSource;


@Configuration  pyj-0313 +1 *
public class DataSourceConfig {

    @Bean  pyj-0313
    public DataSource dataSource2(){
        BasicDataSource dataSource = new BasicDataSource(); // commons-dbcp2 커넥션 풀 생성
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver"); // My SQL 드라이버 설정
        dataSource.setUrl("jdbc:mysql://localhost:3306/testdb"); // DB접속 URL 설정
        dataSource.setUsername("root"); // DB 사용자명
        dataSource.setPassword("1234"); // DB 비밀번호
        return dataSource; // dataSource 반환

    }

    @Bean  pyj-0313
    public DataSource dataSource3(){
        HikariDataSource dataSource = new HikariDataSource(); // Hikari 커넥션 풀 생성
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver"); // My SQL 드라이버 설정
        dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/testdb"); // DB접속 URL 설정
        dataSource.setUsername("root"); // DB 사용자명
        dataSource.setPassword("1234"); // DB 비밀번호
        return dataSource; // dataSource 반환
    }
}
```

DataSourceConfig

@Configuration 클래스가 Spring 설정 클래스임을 나타냄
@Bean 메서드가 반환하는 객체를 Spring 컨테이너에 빈으로 등록
commons-dbcp2의 BasicDataSource를 생성해서 Spring 빈으로 등록
MySQL 드라이버, URL, 계정 정보 설정 후 반환
HikariCP의 HikariDataSource를 생성해서 Spring 빈으로 등록,
dataSource2와 동일한 DB에 연결하지만 다른 커넥션 풀 라이브러리 사용

dataSource2는 setUrl(), dataSource3는 setJdbcUrl()을
사용한다는 점만 다르고 나머지는 동일

03 GlobalExceptionHandler - 박한결

```java
package com.example.demo.Controller.GlobalException;

import lombok.extern.slf4j.Slf4j;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import java.io.FileNotFoundException;

// TODO: 전역 예외 핸들러 3개를 작성하라.
//  필요한 import : org.springframework.ui.Model,
//                  org.springframework.web.bind.annotation.ExceptionHandler, java.io.S
@Slf4j
@ControllerAdvice
```

전역 예외 핸들러 - import 작성

@Slf4j : 로그 출력용 log 객체를 자동 생성하는 Lombok 어노테이션

Model : Controller에서 View로 데이터를 전달하는 객체

@ControllerAdvice : 여러 Controller에서 발생하는 예외를 전역적으로 처리하는 클래스임을 나타내는 어노테이션

@ExceptionHandler : 특정 예외가 발생했을 때 실행할 예외 처리 메서드를 지정하는 어노테이션

03 GlobalExceptionHandler - 박한결

```java
public class GlobalExceptionHandler {

    // TODO 1) @ExceptionHandler(FileNotFoundException.class) -> model 에 담고 "global/error1" 반환
    // FileNotFoundException: 미존재 파일 예외 처리
    // 시스템에서 파일을 찾을 수 없을 때 발생하는 예외를 처리한다.
    @ExceptionHandler(FileNotFoundException.class)
    public String exceptionHandler_1(Exception e, Model model){
        log.error("[Global] ExceptionTestController ex1 : " + e);   // 에러로그 출력
        model.addAttribute( attributeName: "e",e);  // 에러 정보를 view에 전달
        return "global/error1"; //  에러 페이지로 이동
    }

    // TODO 2) @ExceptionHandler(ArithmeticException.class)   -> "global/error2" 반환
    // ArithmeticException: 산술 연산 예외 처리
    // 수학적 계산 오류가 생겼을 때 발생하는 예외를 처리한다.
    @ExceptionHandler(ArithmeticException.class)
    public String exceptionHandler_2(Exception e, Model model){
        log.error("[Global] ExceptionTestController ex2 : " + e);   // 에러로그 출력
        model.addAttribute( attributeName: "e",e);  // 에러 정보를 view에 전달
        return "global/error2"; //  에러 페이지로 이동
    }

    // TODO 3) @ExceptionHandler(Exception.class)            -> "global/error3" 반환
    // Exception: 기타 모든 예외 처리
    // 위에 명시되지 않은 나머지 모든 예외를 처리한다.
    @ExceptionHandler(Exception.class)
    public String exceptionHandler_All(Exception e, Model model){
        log.error("[Global] etc exceptions : " + e);   // 에러로그 출력
        model.addAttribute( attributeName: "e",e);  // 에러 정보를 view에 전달
        return "global/error3"; //  에러 페이지로 이동
    }
```

전역 예외 핸들러 - 예외처리

exceptionHandler_1 FileNotFoundException: 미존재 파일 예외 처리
ArithmeticException: 산술 연산 예외 처리
Exception: 기타 모든 예외 처리

각 예외에 대한 에러 로그를 출력하고, 에러 정보를 view에 전달한 뒤 에러 페이지로 이동시킨다.

04 MemberController - 박영준

```java
@Controller  1gyeolpark +1 *
@Slf4j
@RequestMapping("/member")
public class MemberController {

    @Autowired
    private MemberDAO memberDAO;

    @ExceptionHandler  pyj-0313
    public String SQLExceptionHandler(Exception e, Model model){
        log.error("MEMO SQLEXCEPTION.." + e); // 예외 내용 로그 출력
        model.addAttribute( attributeName: "ex",e.getMessage()); // 예외 메시지를 모델에 담아 뷰로 전달
        return "member/error"; // 에러 뷰로 이동
    }
```

```java
    if (bindingResult.hasErrors()) { // 유효성 검증 오류 확인
        for (FieldError error : bindingResult.getFieldErrors()) { // 오류 필드 순회
            log.info("Error Field : " + error.getField() + " Error Message : " + error.getDefaultMessage()); // 오류 로그 출력
            model.addAttribute(error.getField(), error.getDefaultMessage()); // 오류 메시지 모델에 담기
        }
        return "member/add"; // 유효성 오류 시 입력 폼으로 반환
    }
    int result = memberDAO.insert(memberDTO); // 회원 DB 등록
    if(result>0)
        redirectAttributes.addFlashAttribute( attributeName: "message", attributeValue: "회원등록 성공!"); // 성공 메시지 저장
    return "redirect:/member/list"; // 회원 목록으로 리다이렉트
}
```

MemberController

@ExceptionHandler

DAO 등에서 발생한 예외를 가로채서 처리

예외 메시지를 "ex" 키로 모델에 담아 "member/error" 뷰로 반환

memberAddPost - 유효성 검증

bindingResult.hasErrors()로 오류 여부 확인

오류가 있으면 각 필드의 오류 메시지를 모델에 담고 "member/add" 뷰로 반환

memberAddPost - 회원등록

검증 통과 시 memberDAO.insert(memberDTO)로 DB에 회원 등록

등록 성공 시 "회원등록 성공!" 메시지를 플래시로 저장 한 후

"redirect:/member/list"로 이동

Miri Company

05 MemberDAO - 원병선

```java
public int insert(MemberDTO dto) throws SQLException {  0개의 사용위치   byeongseon8-arch +1
    // insert 메서드를 사용하여 MTO 객체를 DB에 넣음
    try(
            Connection conn = dataSource3.getConnection();
            // dataSource3 객체로부터 Connection 을 하나 얻어옴
            PreparedStatement pstmt = conn.prepareStatement( sql: "insert into tbl_member values(null,?,?,?,?)")
            // PreparedStatement 객체를 생성
    ){
        pstmt.setString( parameterIndex: 1,dto.getName());
        pstmt.setString( parameterIndex: 2,dto.getEmail());
        pstmt.setString( parameterIndex: 3,dto.getPhone());
        pstmt.setTimestamp( parameterIndex: 4, Timestamp.valueOf(LocalDateTime.now()));
        // 각 순서 별 ? 자리에 이름, 이메일, 전화번호를 셋팅

        int result =pstmt.executeUpdate();
        return result;
        // SQL 쿼리문을 실행한 후에 메서드 종료
    }
}
```

MemberDAO

- @Autowired 어노테이션을 통해 멤버 변수에 맞는 Bean을 자동으로 찾아와서 연결
- 메서드를 통해 성공한 행(Row)의 개수를 int 타입으로 반환- try 를 통해 자동으로 자원을 반환하고 빈 자리에 데이터를 채워 넣음
- Query 문을 실행하고 결과를 반환

05 MemberDAO - 원병선

```java
public List<MemberDTO> selectAll() throws SQLException {  1개 사용 위치  byeongseon8-arch +1 *

    try(
            Connection conn = dataSource3.getConnection();
            // dataSource3 에서 Connection을 가져옴
            PreparedStatement pstmt = conn.prepareStatement( sql: "select * from tbl_member order by id desc");
            ResultSet rs = pstmt.executeQuery();
            // 전체 회원을 조회하는 SQL문 실행하여 ResultSet 객체에 담음
    ){
        List<MemberDTO> list = new ArrayList<>();
        // 회원 객체들을 담아 리턴할 리스트 생성
        MemberDTO dto = null;
        // 각 행에 데이터를 담아둘 MemberDTO 참조 변수를 선언
        while(rs.next()){
            // 데이터가 있는 동안 while 문이 반복
            dto = MemberDTO.builder()
                    .id(rs.getLong( columnLabel: "id"))
                    .name(rs.getString( columnLabel: "Name"))
                    .email(rs.getString( columnLabel: "Email"))
                    .phone(rs.getString( columnLabel: "Phone"))
                    .createAt(rs.getTimestamp( columnLabel: "createAt").toLocalDateTime())
                    .build();
            list.add(dto);
            // 셋팅된 값들을 바탕으로 최종적인 MemberDTO 객체 를 완성
        }
        return list;
    }
```

MemberDAO

- MemberDTO를 담은 List 배열 반환
- 실행할 SQL문을 준비하고 order by id desc 코드를 통해 역순 정렬
- select 문을 실행하고 ResultSet 객체에 담음
- 결과를 담을 리스트를 생성하고 MemberDTO dto = null;을 통해 임시로 담을 참조 변수를 선언
- while을 통해 읽을 데이터가 없을 때까지 반복
- builder 를 사용하여 MemberDTO 객체를 생성하고 "id", "Name", "Email", "Phone" 컬럼의 값을 타입에 맞게 가져옴 / 자바의 최신 날짜 타입인 LocalDateTime 으로 변환하여 매핑
- 회원 정보가 담긴 List를 메서드 외부로 반환

06 MemberDTO - 김선길

README.md | MemberDTO.java

```java
package com.example.demo.Domain.Common.Dtos;

// TODO: Lombok( @Data @NoArgsConstructor @AllArgsConstructor @Builder ) 과
//       검증( jakarta.validation.constraints.NotBlank, Email ) import 를 추가하라.

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
```

1. Lombok 애너테이션(@Data, @NoArgsConstructor, @AllArgsConstructor, @Builder)과 검증용 애너테이션(@NotBlank, @Email)을 사용하기 위해 필요한 import 문 추가

2. Lombok을 통해 DTO의 Getter/Setter, 생성자, Builder 패턴을 자동 생성하도록 설정

3. Validation 애너테이션을 활용하여 필수 입력값과 이메일 형식을 검증할 수 있도록 준비

06 MemberDTO - 김선길

```
// TODO: 아래 클래스에 Lombok 애너테이션을 붙여 getter/setter/builder 가 생성되도록 하라.
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberDTO {
```

1. Lombok의 @Data, @NoArgsConstructor, @AllArgsConstructor, @Builder 애너테이션을 클래스에 추가

2. Getter/Setter, 기본 생성자, 전체 필드 생성자, Builder 패턴 코드 자동 생성

06 MemberDTO - 김선길

```
    // TODO: 각 필드에 알맞은 검증 애너테이션을 추가하라.
    private Long id;                    // 식별자 (검증 없음, DB AUTO_INCREMENT)
    @NotBlank(message = "이름은 필수입니다.")
    private String name;                // TODO: @NotBlank
    @NotBlank(message = "이메일은 필수입니다.")
    @Email(message = "올바른 이메일 형식이어야 합니다.")
    private String email;               // TODO: @NotBlank + @Email
    @NotBlank(message = "전화번호 입력은 필수입니다.")
    private String phone;               // TODO: @NotBlank
    private LocalDateTime createAt;     // 등록일시 (검증 없음)

}
```

1. name, email, phone 필드에 @NotBlank를 적용하여 필수 입력 값으로 설정

2. email 필드에는 추가로 @Email을 적용하여 이메일 형식으로 검증

3. id와 createAt은 시스템에서 관리되는 값이므로 별도의 검증 애너테이션을 적용하지 않음